1-1. 윈도우 실행파일 구조(PE파일) 개요

1. 윈도우 실행파일(PE파일) 개요

1.1 PE파일이란?

PE(Portable Executable)파일은 윈도우 실행파일이라고 부르며 윈도우OS에서 사용되는 실행파일형식을 의미하며 UNIX의 COFF(Common Object File Format)을 기반으로 만들어졌다.

- windows 환경에 따라 다르게 부르는데 32bit환경에서는 PE32, 64bit환경에서는 PE+또는 PE32+라고 불린다.
- Linux나 Unix와 같은 계열의 환경에서는 PE파일과 비슷한 형식으로 ELF(Executable and Linkable Format)파일이 존재한다.
- PE파일을 통해서 프로그램이 사용하는 API 또는 DLL 등 다양한 정보와 어느 메모리 주소에 로딩되는지 확인할 수 있다.

1.2 PE파일 종류

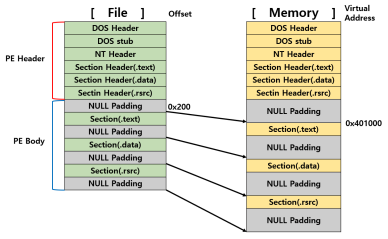
1) 실행 계열 : EXE, SCR

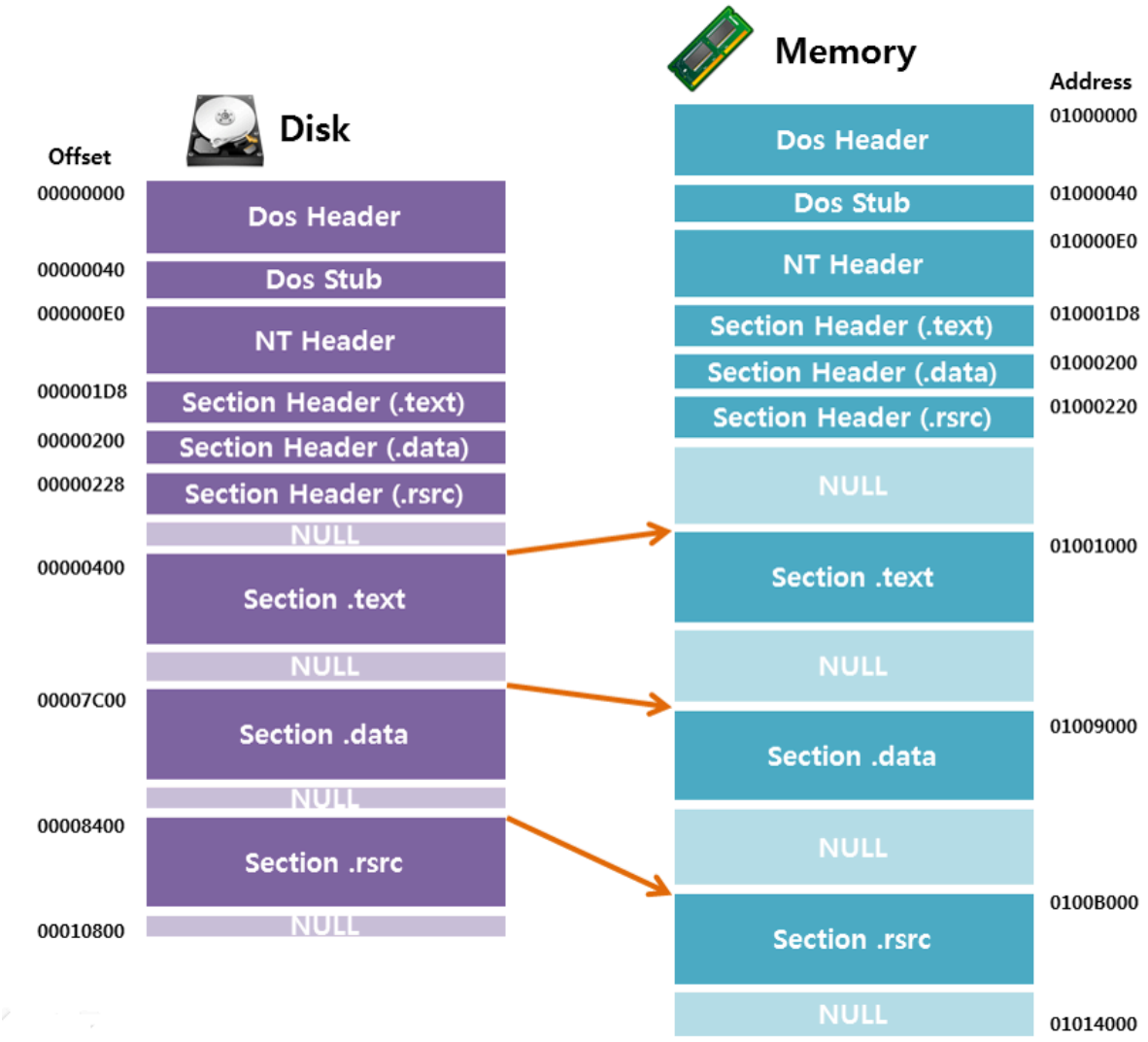
2) 라이브러리 계열 : DLL, OCX, CPL, DRV

3) 드라이버 계열 : SYS, VXD

4) 오브젝트 파일 계열 : OBJ

2. PE파일 구조





1) PE 구조

- 다양한 정보들이 PE Header에 구조체 형식으로 저장
- DOS Header부터 Section Header까지 PE Header이며 밑에 Section들을 합쳐서 PE Body
- 파일에서는 offset, 메모리에서는 VA(Virtual Address)로 위치를 표현

2) Offset, VA, RVA

파일에서는 Offset으로 메모리에 적재될 때는 VA로 위치를 표현

- **Offset** : 파일의 첫 바이트 부터 거리를 뜻함
- **VA(Virtual Address)** : 프로세스 가상 메모리의 절대주소
- **RVA(Relative Virtual Address)** : 기준(ImageBase)로부터 상대주소
- **VA = RVA + ImageBase**

PE Header에 많은 정보는 RVA형태로 된 것을이 존재

- **이유 : PE파일(주로 DLL)이 가상 메모리의 특정 위치에 로딩될 때 해당 위치에 다른 PE 파일(DLL)이 로딩되어 있을 수 있으므로 Relocation과정을 통해 빈 공간에 로딩되어야 한다.** 만약, VA(절대주소)로 되어있다면 정상적인 액세스가 이뤄지지 않지만 RVA(상대주소)로 되어있다면 Relocation이 발생해도 ImageBase에 대한 상대주소는 변하지 않기 때문에 문제 없이 액세스가 가능하다.

3) Null Padding

- 헤더의 끝 부분과 섹션별로 사이사이에 Null값이 존재
- **파일 또는 메모리에서 섹션의 시작위치는 각각 최소 기본 단위의 배수에 해당하는 위치여야하므로 빈공간은 Null**로 채워버림
- 이를 Alignment라 부르며 여러 가지 내부 연산 등 처리상의 효율성을 위해 특정 단위로 간격을 맞추며 빈 공간은 Null로 채우는 것이다.
-

2.1 PE Header

1) DOS Header

- DOS 파일에 대한 하위 호환성을 고려하여 만듦
- **구조체** : **IMAGE_DOS_HEADER**
- **주요 멤버**

1-1) e_magic : DOS signature(매직넘버)로 PE파일을 나타내는 첫 2바이트, MZ(5A4D0로 고정되어 있으며 MZ이 아니면 로더는 해당 파일을 PE로 인식하지 않는다. (참고로 MZ은 DOS 실행 파일을 설계한 Mark Zbikowski의 이니셜)

1-2) e_lfanew : NT Header가 시작되는 위치의 옵셋을 뜻하며 e_lfanew값이 가리키는 위치에 NT Header구조체가 존재해야 한다.

2) DOS Stub

- DOS환경에서 실행되는 코드를 가진 영역이며 일종의 옵션으로 DOS Stub이 없어도 파일 실행에 문제가 없다.
- 코드와 데이터의 혼합으로 이뤄졌으며 DOS환경에서 실행하면 "This program cannot be run in DOS mode"문자열을 출력하고 종료

3) NT Header

- **구조체 : IMAGE_NT_HEADERS**
- DOS Header의 멤버 중 e_lfanew에 저장된 주소 값이 이 부분을 가리킴

- **NT Header는 파일 실행에 필요한 정보가 저장**
- NT Header의 구조체에는 총 3개의 멤버로 구성되어있으며 다음과 같다.

3-1) Signature

- 제일 첫 멤버로 일반적으로 50450000h("PE"00)을 가진다.(변경 불가)

3-2) File Header

- **구조체 : IMAGE_FILE_HEADER**
- 파일의 개략적인 속성을 나타냄
- **주요 멤버(해당 값들이 정확히 세팅되지 않으면 파일은 정상적으로 실행되지 않음)**

① **Machine** : 파일의 실행 대상 플랫폼을 나타냄(winnt.h에 정의됨) I386(0x014C), IA64(0x200), AMD64(0x8664)

② **NumberOfSections** : 파일에 존재하는 섹션의 개수(파일에 따따라 섹션의 개수가 달라지므로)이며 반드시 0보다 커야함, (정의된 섹션 개수 > 실제 섹션 개수 : 실행 에러), (정의된 섹션 개수 < 실제 섹션 개수 : 정의된 개수만큼 인식)

③ **SizeOfOptionalHeader** : 다음에 이어지는 Optional Header의 크기를 알려줌

④ **Characteristics :** 파일의 속성을 나타내는 값으로 실행 파일인지 DLL파일인지 등의 정보들이 bit OR연산을 통해 최종 값으로 설정된다. (winnt.h파일에 정의됨), exe파일(0x0002), dll파일(0x2000)

⑤ **TimeDateStamp** : 파일의 실행에 영향을 미치지 않는 값으로, 해당 파일의 빌드 시간을 나타냄

3-3) Optional Header

- **구조체 : IMAGE_OPTIONAL_HEADER**
- **파일 실행에 필요한 주요 정보들을 저장**
- **주요 멤버**

① **Magic** : IMAGE_OPTIONAL_HEADER32인지 64인지 구분(32인 경우 0X10B, 64인 경우 0x20B)

② **AddressOfEntryPoint** : 파일이 메모리에 매핑된 후 코드 시작 주소이며 ImageBase 값에 이 값을 더해 코드 시작 지점을 설정(즉, Entry Point의 RVA를 가짐)

③ **ImageBase** : PE파일이 로딩되는 시작 주소이며 EXE, DLL파일은 user memory영역에 위치하며 SYS파일은 kernel memory영역에 위치한다. 일반적으로 EXE파일은 0x400000, DLL파일은 0x1000000의 값을 가진다.

④ **SectionAlignment, FileAlignment** : 메모리/파일에서 섹션의 최소단위를 나타내며 각 섹션은 반드시 Alignment의 배수여야 한다.(섹션에서 크기 남더라도 0으로 채워서 Alignment값을 맞춤)

⑤ **SizeOfImage** : 메모리에 로딩될 때 가상 메모리에서 PE Image가 차지하는 크기

⑥ **SizeOfHeader** : PE Header의 전체 크기(FileAlignmaet의 배수여야 한다.)

⑦ **Subsystem** : 동작환경을 정의한 것으로 sys파일(0x1), GUI파일(0x2), CLI(0x3)의 값으로 세팅

⑧ **NumberOfRvaAndSize** : DataDirectory배열의 개수로 일반적으로 16개의 디렉터리를 가지지만 해당 멤버로 디렉터리 수를 정할 수 있다.

⑨ **DataDirectory** : **IMAGE_DATA_DIRECTORY**구조체의 배열로써 디렉토리별로 각각의 정보를 담음

- Export Directory : DLL등의 파일에서 외부에 함수를 공개하기 위한 정보들을 가짐
- Import Directory : 프로그램 실행을 위해 Import하는 DLL 이름과 사용할 함수 정보가 담긴 INT주소와 IAT주소 같은 정보가 담김
- TLS Directory : 스레드 지역 저장소 초기화 섹션에 대한 포인터
- IAT Directory : IAT의 시작번지를 가리킴

4) Section Header

- 각 Section의 헤더들이 존재하며 Section들은 메타데이터를 저장하고 메모리에 로드될 때 필요한 정보를 포함하고 있다.
- **주요 멤버**

4-1) VirtualSize : 메모리에서 섹션의 크기

4-2) VirtualAddress : 메모리에서 섹션의 시작주소(RVA)

4-3) SizeOfRawData : 파일에서 섹션의 크기

4-4) PointerToRawData : 파일에서 섹션의 Offset

4-5) Characteristics : 섹션의 속성, Bit OR연산

2.2 Section

1) Section유형

1-1) Code

.text : 프로그램 실행 코드를 담고 있는 섹션

1-2) Data

.data : 읽고 쓰기가 가능한 데이터 섹션으로 초기화된 전역변수와 정적 변수 등이 위치

.rdata : 읽기 전용 데이터 섹션으로 상수형 변수, 문자열 상수 등이 위치

.bss : 초기화되지 않은 전역변수를 담고 있는 섹션

1-3) Import API

.idata : Import할 DLL과 API에 대한 정보를 담고 있는 섹션으로 대표적으로 IAT존재

.didat : Delay-loading importDLL 정보(프로그램 실행 후 DLL사용시에만 연결하며 릴리즈 모드 시 다른 섹션에 병합)

1-4) Export API

.edata : Export할 API정보를 담고 있는 섹션(주로 DLL에서 사용되며 .text 또는 .rdata에 병합)

1-5) Resource

.rsrc : 리소스 관련 데이터를 담고 있는 섹션(ex) 아이콘, 커서 등)

1-6) 재배치 정보

.reloc : 기본 재배치 정보를 담고 있는 섹션(주로 DLL에서 사용)

1-7) TLS

.tls : 스레드 지역 저장소

1-8) Debugging

.debug$p : 미리 컴파일된 헤더 사용시에만 존재